

Database Systems

Name: Kamal Zhanserik

Laboratory Work, Week 2: Relational Model & Keys

Part 1: Key Identification

Task 1.1 – Relation A: Employee

Employee(EmpID, SSN, Email, Phone, Name, Department, Salary)

1. Superkeys

A superkey is any set of attributes that uniquely identifies a row.

- 1 {EmpID}
- 2 {SSN}
- 3 {Email}
- 4 {EmpID, Name}
- 5 {SSN, Department}
- 6 {Email, Salary}
- 7 {EmpID, SSN}
- 8 {EmpID, SSN, Email, Phone, Name, Department, Salary} (all attributes)

Any set that contains EmpID, SSN or Email is a superkey.

2. Candidate keys

A candidate key is a minimal superkey (no attribute can be removed): **{EmpID}**, **{SSN}**, **{Email}**.

Phone is unique in the sample data, but I don't consider it a candidate key (see question 4). Name, Department and Salary are not unique: for example, two employees can both work in IT or have the same name.

3. Primary key choice

I choose **EmpID** because:

- it is short and numeric, so it is efficient for indexing and for foreign keys;
- it is assigned by the company and does not change;
- SSN is sensitive personal data and should not be copied into other tables as a foreign key;
- Email can change (for example, if the name or domain changes).

4. Can two employees have the same phone number?

In the sample data all phone numbers are different, but three rows cannot prove that a rule always holds. A sample can show that an attribute is *not* unique, but it can never prove that it *is* unique. In real life two employees can share a phone, for example a department line or a shared office extension. So yes, it is possible, and Phone should not be used as a key.

Task 1.1 – Relation B: Registration

Registration(StudentID, CourseCode, Section, Semester, Year, Grade, Credits)

1. Minimum primary key

(StudentID, CourseCode, Section, Semester, Year)

2. Why each attribute is needed

Attribute	Reason
StudentID	Many students take the same section, so without it we don't know who registered.
CourseCode	One student takes several courses in the same semester.
Section	The business rule is about “the same course section”, so the section is part of what makes a registration unique.
Semester	A student can take the same course in different semesters (for example, retaking it).
Year	“Fall” alone is not enough: Fall 2024 and Fall 2025 are different semesters.

Grade and Credits are not part of the key, because they describe the registration and do not identify it.

3. Additional candidate keys

There are no other candidate keys. Grade and Credits can repeat, so they cannot replace any key attribute.

Note: Credits depends only on (CourseCode, Section, Semester, Year), not on StudentID. This is a partial dependency, so the table is not in 2NF. Credits should be moved to a separate CourseSection table.

Task 1.2 – Foreign Key Design

Foreign key	References	Meaning
Student.AdvisorID	Professor.ProfID	Each student's advisor is a professor
Course.DepartmentCode	Department.DeptCode	Each course belongs to a department
Department.ChairID	Professor.ProfID	The department chair is a professor
Professor.Department	Department.DeptCode	A professor works in a department
Enrollment.StudentID	Student.StudentID	The enrolled student must exist
Enrollment.CourseID	Course.CourseID	The course must exist

Notes:

- The primary key of Enrollment is **(StudentID, CourseID, Semester)**.
- Professor.Department is a foreign key only if it stores the department code. If it stores the name, it should be changed to DeptCode.
- Student.Major could also reference Department.DeptCode if majors are defined by departments. I left it as a normal attribute because a major is not always the same as a department.

- There is a circular reference between Professor and Department (Professor → Department and Department.ChairID → Professor). ChairID should allow NULL, so that a new department can be inserted before a chair is assigned.

Part 2: ER Diagram Construction

Notation (Chen): rectangle = strong entity, double rectangle = weak entity; oval = attribute, double oval = multi-valued, dashed oval = derived; dashed underline = partial key of a weak entity; diamond = relationship, double diamond = identifying relationship; underline = primary key; double line = total participation, single line = partial participation.

Task 2.1 – Hospital Management System

1. Entities

Entity	Type	Reason
Patient	Strong	Has its own PatientID
Doctor	Strong	Has its own DoctorID
Department	Strong	Has its own DeptCode
Appointment	Strong	Identified by AppointmentID
Prescription	Strong	Identified by PrescriptionID
HospitalRoom	Weak	Room number is only unique inside a department (room 101 exists in Cardiology and in Neurology)

2. Attributes

Entity	Attributes (type)
Patient	<u>PatientID</u> (simple, PK), Name (simple), BirthDate (simple), Age (derived from BirthDate), Address (composite: Street, City, State, Zip), Phone (multi-valued), Insurance (composite: Provider, PolicyNumber)
Doctor	<u>DoctorID</u> (simple, PK), Name (simple), Specialization (multi-valued), Phone (simple), OfficeLocation (simple)
Department	<u>DeptCode</u> (simple, PK), DeptName (simple), Location (simple)
Appointment	<u>AppointmentID</u> (simple, PK), DateTime (composite: Date, Time), Purpose (simple), Notes (simple)
Prescription	<u>PrescriptionID</u> (simple, PK), Medication, Dosage, Instructions, DatePrescribed (all simple)
HospitalRoom	RoomNumber (partial key), RoomType (simple). Full key: (DeptCode, RoomNumber)

3. Relationships

Relationship	Entities	Cardinality	Participation
WorksIn	Doctor – Department	N:1	Doctor total, Department partial
Contains (identifying)	Department – HospitalRoom	1:N	HospitalRoom total
Books	Patient – Appointment	1:N	Appointment total
Attends	Doctor – Appointment	1:N	Appointment total
Writes	Doctor – Prescription	1:N	Prescription total
Receives	Patient – Prescription	1:N	Prescription total

Design note: Appointment could also be modeled as an M:N relationship between Patient and Doctor. I made it an entity because the same patient can see the same doctor many times, and each visit needs its own date, purpose and notes.

4-5. ER Diagram

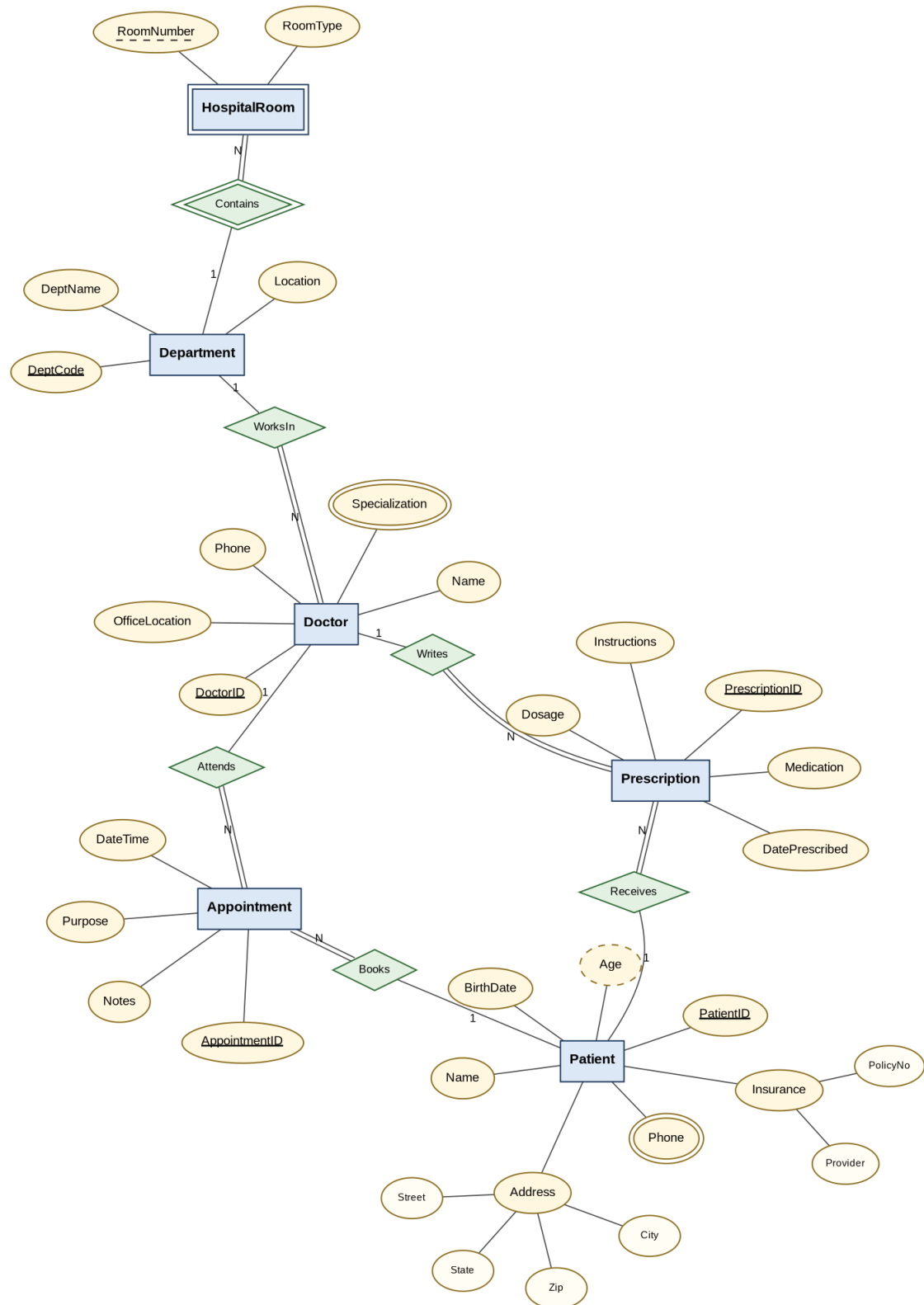


Figure 1. ER diagram – Hospital Management System

Task 2.2 – E-commerce Platform

Entities and attributes

Entity	Attributes
Customer	<u>CustomerID</u> , Name, Email, Phone, BillingAddress (composite: Street, City, Zip, Country)
ShippingAddress	<u>AddressID</u> , RecipientName, Street, City, Zip, Country
Order	<u>OrderID</u> , OrderDate, Status, TotalAmount (derived)
OrderItem (weak)	LineNumber (partial key), Quantity, PriceAtOrder
Product	<u>ProductID</u> , Name, Description, CurrentPrice
Category	<u>CategoryID</u> , CategoryName
Vendor	<u>VendorID</u> , VendorName, ContactEmail, Phone
Inventory	<u>InventoryID</u> , QuantityInStock, ReorderLevel, LastUpdated

Relationships

Relationship	Entities	Cardinality	Attributes
Places	Customer – Order	1:N	–
HasAddress	Customer – ShippingAddress	1:N	–
ShipsTo	Order – ShippingAddress	N:1	–
Contains (identifying)	Order – OrderItem	1:N	–
RefersTo	OrderItem – Product	N:1	–
BelongsTo	Product – Category	N:1	–
Supplies	Vendor – Product	M:N	SupplyPrice
Reviews	Customer – Product	M:N	Rating, Comment, ReviewDate
Tracks	Product – Inventory	1:1	–

Billing address is a composite attribute of Customer, while shipping addresses are a separate entity. This way one customer can have several shipping addresses that differ from the billing address.

Weak entity: OrderItem

- LineNumber (1, 2, 3...) is unique only inside one order, because every order starts again from line 1;
- an order item cannot exist without its order;
- its full key includes the owner's key: (**OrderID**, **LineNumber**).

M:N relationship with attributes: Reviews (Customer – Product)

One customer can review many products, and one product can be reviewed by many customers. Rating, Comment and ReviewDate don't belong to the customer alone or to the product alone – they describe a specific customer's opinion of a specific product, so they are attributes of the relationship. Supplies (Vendor – Product) with SupplyPrice is a second example, because the same product can be bought from different vendors at different prices.

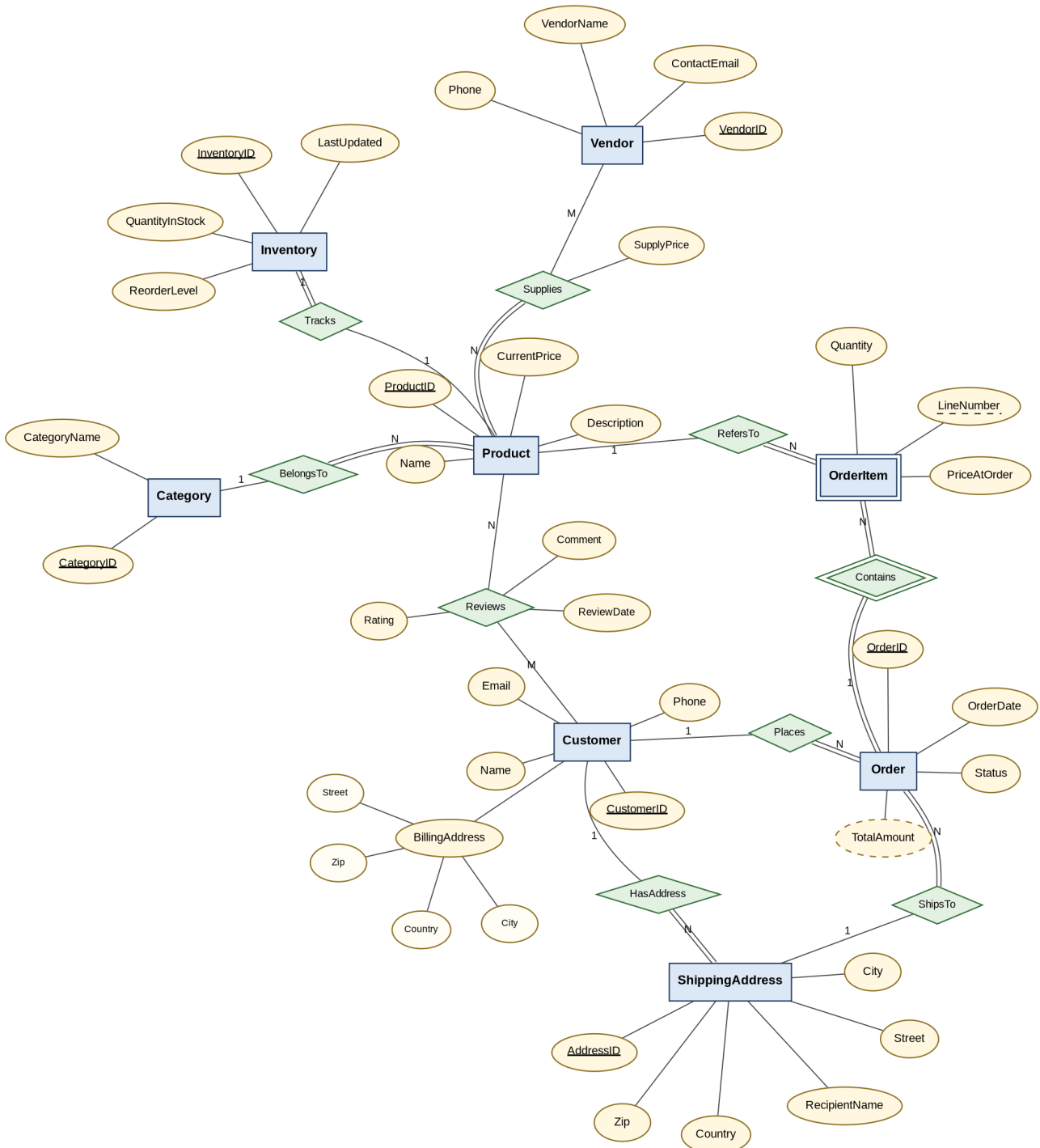


Figure 2. ER diagram – E-commerce Platform

Part 4: Normalization Workshop

Task 4.1 – StudentProject

StudentProject(StudentID, StudentName, StudentMajor, ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept, Role, HoursWorked, StartDate, EndDate)

Assumptions: each project has one supervisor; a student can work on several projects; Role, HoursWorked, StartDate and EndDate describe one student's participation in one project.

1. Functional dependencies

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID
- SupervisorID → SupervisorName, SupervisorDept
- (StudentID, ProjectID) → Role, HoursWorked, StartDate, EndDate

2. Problems

Redundancy: a student's name and major are repeated in every project row of that student; project title, type and all supervisor information are repeated for every student in the project.

Update anomaly: if supervisor S01 moves from the CS department to IT, SupervisorDept must be changed in every row of every project S01 supervises. If one row is missed, the data becomes inconsistent.

Insert anomaly: a new project cannot be added until at least one student joins it, because StudentID is part of the primary key and cannot be NULL. The same is true for a new supervisor with no projects.

Delete anomaly: if the only student on project P05 leaves and the row is deleted, we also lose the project title, type and supervisor information.

3. 1NF

The table is in 1NF: each attribute holds one atomic value and there are no repeating groups. A violation would appear if, for example, Role stored several values in one cell (“Developer, Tester”). The fix would be to store one role per row or move roles to a separate table.

4. 2NF

Primary key: (**StudentID, ProjectID**)

Partial dependencies (depend on only one part of the key):

- StudentID → StudentName, StudentMajor
- ProjectID → ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept

2NF decomposition:

Student(StudentID, StudentName, StudentMajor)

Project(ProjectID, ProjectTitle, ProjectType, SupervisorID, SupervisorName, SupervisorDept)

StudentProject(StudentID, ProjectID, Role, HoursWorked, StartDate, EndDate)

5. 3NF

Transitive dependency in Project: $\text{ProjectID} \rightarrow \text{SupervisorID} \rightarrow \text{SupervisorName}$, SupervisorDept . SupervisorName and SupervisorDept depend on a non-key attribute, so they are moved to their own table.

Final 3NF schema (PK underlined, FK in italics):

Student(StudentID, StudentName, StudentMajor)
Supervisor(SupervisorID, SupervisorName, SupervisorDept)
Project(ProjectID, ProjectTitle, ProjectType, *SupervisorID* $\rightarrow$ Supervisor)
StudentProject(StudentID $\rightarrow$ Student, ProjectID $\rightarrow$ Project, Role, HoursWorked, StartDate, EndDate)

Table definitions with data types:

Table	Columns and data types	Keys
Student	StudentID INT, StudentName VARCHAR(100) NOT NULL, StudentMajor VARCHAR(50)	PK: <u>StudentID</u>
Supervisor	SupervisorID INT, SupervisorName VARCHAR(100) NOT NULL, SupervisorDept VARCHAR(50)	PK: <u>SupervisorID</u>
Project	ProjectID INT, ProjectTitle VARCHAR(150) NOT NULL, ProjectType VARCHAR(30), SupervisorID INT NOT NULL	PK: <u>ProjectID</u> FK: <i>SupervisorID</i> $\rightarrow$ Supervisor
StudentProject	StudentID INT, ProjectID INT, Role VARCHAR(30), HoursWorked DECIMAL(6,1), StartDate DATE NOT NULL, EndDate DATE	PK: <u>StudentID</u> , <u>ProjectID</u> FK: <i>StudentID</i> $\rightarrow$ Student, <i>ProjectID</i> $\rightarrow$ Project

All three anomalies are now solved: a project can exist without students, a supervisor's department is stored only once, and deleting a participation row does not delete the project.

Task 4.2 – CourseSchedule

CourseSchedule(*StudentID*, *StudentMajor*, *CourseID*, *CourseName*, *InstructorID*, *InstructorName*, *TimeSlot*, *Room*, *Building*)

Why it is tricky: the table has no SectionID, so a section must be identified by *when* and *where* it happens, not by CourseID alone.

Assumptions (they follow from the rules and from real life): A1 – one room hosts only one section in a given time slot; A2 – an instructor cannot teach two sections in the same time slot.

2. Functional dependencies

- 1 StudentID $\rightarrow$ StudentMajor
- 2 CourseID $\rightarrow$ CourseName
- 3 InstructorID $\rightarrow$ InstructorName
- 4 Room $\rightarrow$ Building (rooms are unique across campus)
- 5 (TimeSlot, Room) $\rightarrow$ CourseID, InstructorID (A1)
- 6 (InstructorID, TimeSlot) $\rightarrow$ CourseID, Room (A2)

1. Primary key

Closure of {StudentID, TimeSlot, Room}: StudentID $\rightarrow$ StudentMajor; Room $\rightarrow$ Building; (TimeSlot, Room) $\rightarrow$ CourseID, InstructorID; CourseID $\rightarrow$ CourseName; InstructorID $\rightarrow$ InstructorName. This gives all 9 attributes, so the set is a key. No attribute can be removed: without StudentID we don't know which student, and without TimeSlot or Room we don't know which section.

Candidate keys: **(StudentID, TimeSlot, Room)** – chosen as primary key, and (StudentID, InstructorID, TimeSlot). If assumption A1 is not accepted, the key becomes (StudentID, CourseID, TimeSlot, Room).

3. BCNF check

BCNF requires the left side of every FD to be a superkey.

FD	Left side a superkey?
StudentID $\rightarrow$ StudentMajor	No
CourseID $\rightarrow$ CourseName	No
InstructorID $\rightarrow$ InstructorName	No
Room $\rightarrow$ Building	No
(TimeSlot, Room) $\rightarrow$ CourseID, InstructorID	No
(InstructorID, TimeSlot) $\rightarrow$ CourseID, Room	No

The table is **not in BCNF**. It is not even in 2NF, because StudentMajor and Building depend on only part of the key.

4. Decomposition to BCNF

Step	Split on FD	New table	Remaining table
1	StudentID → StudentMajor	R1 = Student(StudentID, StudentMajor)	(StudentID, CourseID, CourseName, InstructorID, InstructorName, TimeSlot, Room, Building)
2	CourseID → CourseName	R2 = Course(CourseID, CourseName)	(StudentID, CourseID, InstructorID, InstructorName, TimeSlot, Room, Building)
3	InstructorID → InstructorName	R3 = Instructor(InstructorID, InstructorName)	(StudentID, CourseID, InstructorID, TimeSlot, Room, Building)
4	Room → Building	R4 = Room(Room, Building)	(StudentID, CourseID, InstructorID, TimeSlot, Room)
5	(TimeSlot, Room) → CourseID, InstructorID	R5 = Section(TimeSlot, Room, CourseID, InstructorID)	R6 = Enrollment(StudentID, TimeSlot, Room)

Checking the result: in R5, (TimeSlot, Room) is the key, and (InstructorID, TimeSlot) is also a superkey because it determines Room – so R5 is in BCNF. In R6 all attributes form the key and there are no non-trivial FDs, so R6 is in BCNF.

Final BCNF schema:

Student(StudentID, StudentMajor)
 Course(CourseID, CourseName)
 Instructor(InstructorID, InstructorName)
 Room(Room, Building)
 Section(TimeSlot, Room → Room, *CourseID* → Course, *InstructorID* → Instructor)
 Enrollment(StudentID → Student, TimeSlot, Room); FK (TimeSlot, Room) → Section

Table definitions with data types:

Table	Columns and data types	Keys
Student	StudentID INT, StudentMajor VARCHAR(50) NOT NULL	PK: <u>StudentID</u>
Course	CourseID VARCHAR(10), CourseName VARCHAR(100) NOT NULL	PK: <u>CourseID</u>
Instructor	InstructorID INT, InstructorName VARCHAR(100) NOT NULL	PK: <u>InstructorID</u>
Room	Room VARCHAR(10), Building VARCHAR(50) NOT NULL	PK: <u>Room</u>
Section	TimeSlot VARCHAR(20), Room VARCHAR(10), CourseID VARCHAR(10) NOT NULL, InstructorID INT NOT NULL; UNIQUE (InstructorID, TimeSlot)	PK: <u>TimeSlot</u> , <u>Room</u> FK: <i>Room</i> → Room, <i>CourseID</i> → Course, <i>InstructorID</i> → Instructor

Table	Columns and data types	Keys
Enrollment	StudentID INT, TimeSlot VARCHAR(20), Room VARCHAR(10)	PK: <u>StudentID, TimeSlot, Room</u> FK: <i>StudentID</i> → Student, (<i>TimeSlot, Room</i>) → Section

5. Loss of information

- **Lossless join:** each step split the table on an FD whose left side became the key of the new table. Joining the tables back on those attributes returns exactly the original rows, with no spurious tuples.
- **Dependency preservation:** all six FDs can still be checked inside a single table (FD 1 in Student, FD 2 in Course, FD 3 in Instructor, FD 4 in Room, FDs 5 and 6 in Section). BCNF does not always preserve dependencies, but here it does.
- **Practical limitation:** Enrollment refers to a section by (TimeSlot, Room). If a section moves to another room, the change must cascade to all its enrollments. In a real system I would add a surrogate SectionID.
- **Improvement:** we can now store a course, instructor or room even if no student is enrolled yet, which was impossible in the original table.

Part 5: Design Challenge

Task 5.1 – Student Clubs and Organizations

1. ER Diagram

Entity	Attributes
Student	<u>StudentID</u> , Name (composite: FirstName, LastName), Email, Major, StudyYear
Faculty	<u>FacultyID</u> , Name (composite: FirstName, LastName), Email, Department
Club	<u>ClubID</u> , ClubName, Description, Category, FoundedDate
Event	<u>EventID</u> , Title, Description, StartTime, EndTime
Room	<u>RoomID</u> , Building, RoomNumber, Capacity
Reservation	<u>ReservationID</u> , StartTime, EndTime, Status
OfficerTerm (weak, owner Club)	Position (partial key), TermStart (partial key), TermEnd
Budget (weak, owner Club)	AcademicYear (partial key), AllocatedAmount, RemainingAmount (derived)
Expense	<u>ExpenseID</u> , Amount, ExpenseDate, Description

Relationship	Entities	Cardinality	Attributes
Advises	Faculty – Club	1:N (Club total)	–
MemberOf	Student – Club	M:N	JoinDate, Status
HasOfficer (identifying)	Club – OfficerTerm	1:N (OfficerTerm total)	–

Relationship	Entities	Cardinality	Attributes
Holds	Student – OfficerTerm	1:N (OfficerTerm total)	–
Organizes	Club – Event	1:N (Event total)	–
Attends	Student – Event	M:N	CheckInTime
For	Event – Reservation	1:N	–
ReservedRoom	Room – Reservation	1:N	–
Has (identifying)	Club – Budget	1:N (Budget total)	–
Spends	Budget – Expense	1:N	–
RelatedTo	Event – Expense	1:N (Expense partial)	–

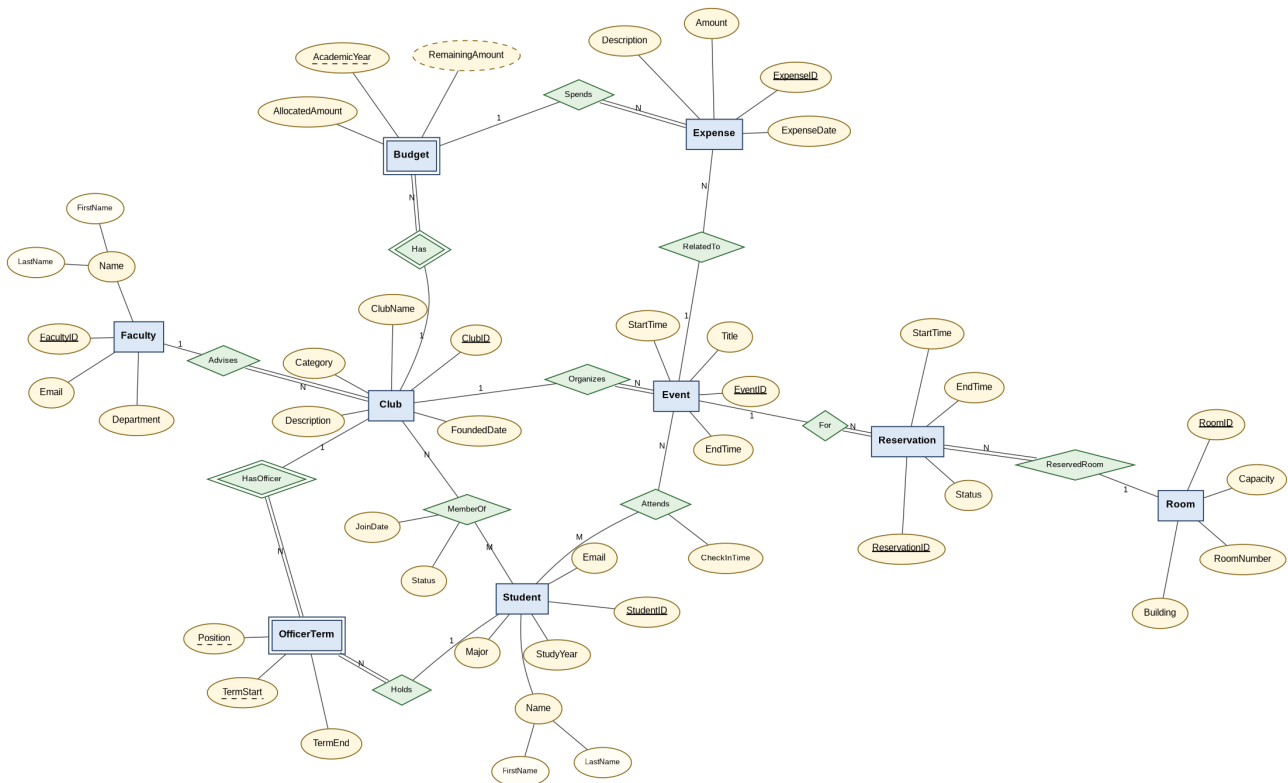


Figure 3. ER diagram – Student Clubs and Organizations

2. Relational Schema (3NF)

PK underlined, FK in italics with the referenced table.

Table	Columns and data types	Keys
Student	StudentID INT, FirstName VARCHAR(50), LastName VARCHAR(50), Email VARCHAR(100) UNIQUE, Major VARCHAR(50), StudyYear SMALLINT	PK: <u>StudentID</u>
Faculty	FacultyID INT, FirstName VARCHAR(50), LastName VARCHAR(50), Email VARCHAR(100) UNIQUE, Department VARCHAR(50)	PK: <u>FacultyID</u>
Club	ClubID INT, ClubName VARCHAR(100) UNIQUE, Description TEXT, Category VARCHAR(50), FoundedDate DATE, AdvisorID INT NOT NULL	PK: <u>ClubID</u> FK: <i>AdvisorID</i> → Faculty

Table	Columns and data types	Keys
Membership	StudentID INT, ClubID INT, JoinDate DATE, Status VARCHAR(20)	PK: <u>StudentID, ClubID</u> FK: <i>StudentID</i> → Student, <i>ClubID</i> → Club
OfficerTerm	ClubID INT, Position VARCHAR(30), TermStart DATE, TermEnd DATE, StudentID INT NOT NULL	PK: <u>ClubID, Position, TermStart</u> FK: (<i>StudentID, ClubID</i>) → Membership
Room	RoomID INT, Building VARCHAR(50), RoomNumber VARCHAR(10), Capacity INT; UNIQUE (Building, RoomNumber)	PK: <u>RoomID</u>
Event	EventID INT, ClubID INT NOT NULL, Title VARCHAR(100), Description TEXT, StartTime TIMESTAMP, EndTime TIMESTAMP	PK: <u>EventID</u> FK: <i>ClubID</i> → Club
Reservation	ReservationID INT, EventID INT, RoomID INT, StartTime TIMESTAMP, EndTime TIMESTAMP, Status VARCHAR(20)	PK: <u>ReservationID</u> FK: <i>EventID</i> → Event, <i>RoomID</i> → Room
Attendance	StudentID INT, EventID INT, CheckInTime TIMESTAMP	PK: <u>StudentID, EventID</u> FK: <i>StudentID</i> → Student, <i>EventID</i> → Event
Budget	ClubID INT, AcademicYear VARCHAR(9), AllocatedAmount NUMERIC(10,2) NOT NULL	PK: <u>ClubID, AcademicYear</u> FK: <i>ClubID</i> → Club
Expense	ExpenseID INT, ClubID INT, AcademicYear VARCHAR(9), EventID INT NULL, Amount NUMERIC(10,2), ExpenseDate DATE, Description VARCHAR(200)	PK: <u>ExpenseID</u> FK: (<i>ClubID, AcademicYear</i>) → Budget, <i>EventID</i> → Event

Why this schema is in 3NF: every non-key attribute depends only on the whole key of its table. The advisor's name and department are stored in Faculty, not repeated in Club; Building is stored in Room, not in Event. RemainingAmount is not stored, because it is calculated as AllocatedAmount minus the sum of Expense.Amount – storing it would create redundancy and a risk of inconsistent values.

3. Design decision

How to store officer positions. There were two valid options:

- **Option A:** add a Position column to the Membership table (NULL for ordinary members).
- **Option B:** create a separate OfficerTerm table.

I chose **Option B** because:

- a student can hold different positions in different years, and Option A would keep only the current one, losing the history;
- Option A would contain many NULLs, since most members are not officers;
- the key (ClubID, Position, TermStart) guarantees that a club has only one president in a given term;
- the foreign key to Membership guarantees that only a club member can be an officer.

The downside is one more table and one more join. I think that is acceptable, because officer history is important for the university.

4. Example queries

- 1 “Show the total expenses of each club for the 2025–2026 academic year and compare them with the club's allocated budget.”
- 2 “List all active members of the Robotics Club who have not attended any of the club's events this semester.”
- 3 “Find all clubs advised by a given faculty member, together with the current president of each club and the number of active members.”